

The Rendering Pipeline: Box vs. Payload Projection

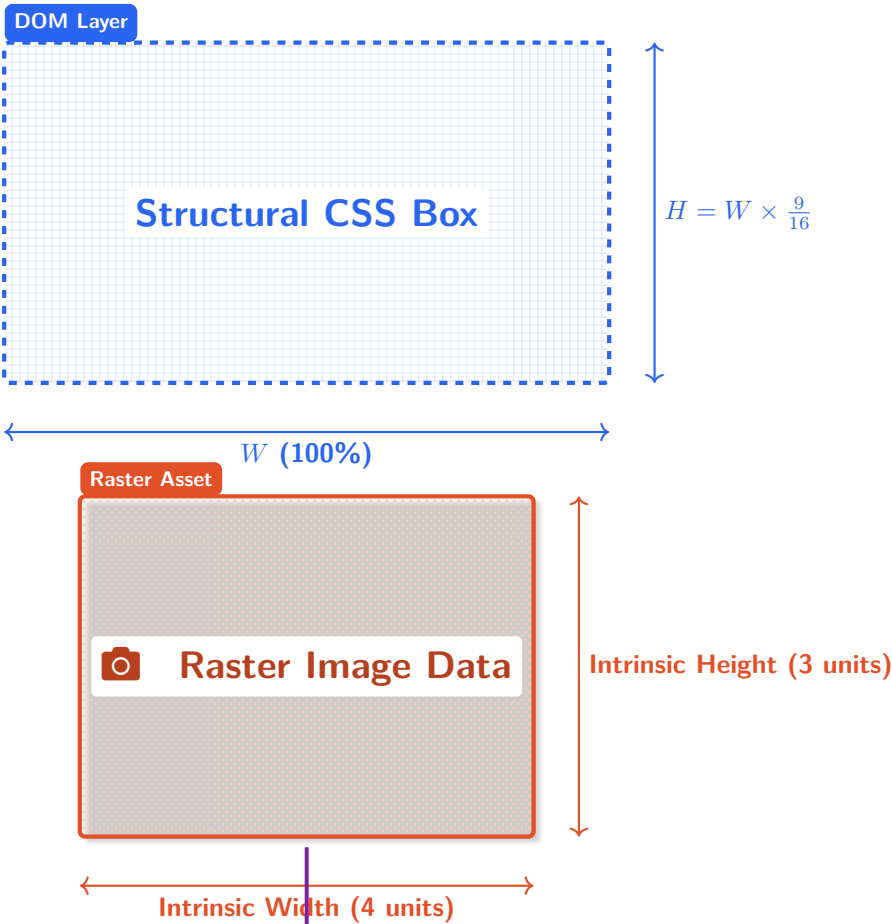
How object-fit maps arbitrary physical intrinsic data into the algorithmically constrained aspect-ratio box

1. The Layout Box (16:9)

Constructed solely from CSS algorithms width: 100% and aspect-ratio: 16 / 9. Exists independently of image content in the layout tree.

2. The Intrinsic Payload (4:3)

The physical image bytes received over the network. It has its own unchangeable intrinsic aspect ratio dictated by the camera/creator (e.g. 4000x3000px).

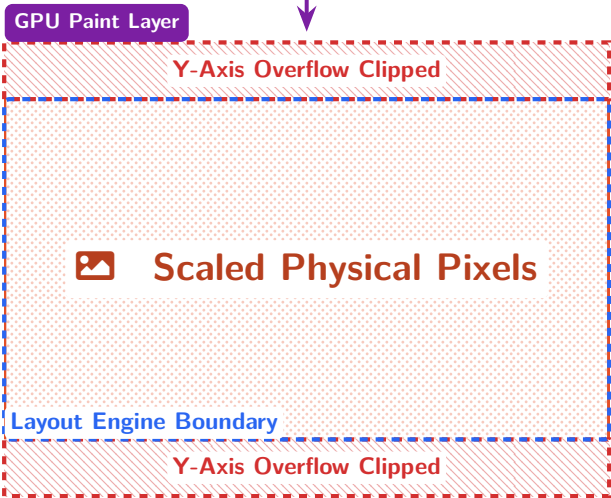


object-fit: cover Transformation Matrix

$$\text{Scale} = \max\left(\frac{W_{\text{box}}}{W_{\text{img}}}, \frac{H_{\text{box}}}{H_{\text{img}}}\right) = \max\left(\frac{8}{6}, \frac{4.5}{4.5}\right) = 1.33\times$$

3. The Final Paint Projection

The browser's graphics layer scales the payload uniformly to cover the structural box, mathematically discarding (clipping) the spatial mismatch. **Layout remains entirely untouched!**



Architectural Independence

By utilizing aspect-ratio paired with object-fit, the **DOM Layout phase** is mathematically severed from the **Paint/Render phase**. The structural HTML container is guaranteed stable by the layout engine (preventing CLS), while the GPU graphics hardware projects the network payload into the predefined coordinates during the Paint phase, seamlessly clipping any aspect ratio inconsistencies.